



SENIOR THESIS IN MATHEMATICS

**Optimization Routines:
Hybridization of the
Levenberg-Marquardt and
Whale Optimization
Algorithms**

Author:
Sam Butler

Advisor:
Dr. Jo Hardin

Submitted to Pomona College in Partial Fulfillment
of the Degree of Bachelor of Arts

May 7, 2026

Abstract

This thesis explores the advantages, disadvantages, and applications of different optimization routines in the context of different surfaces, namely the Levenberg-Marquardt Algorithm and the Whale Optimization Algorithm. We use simulation to evaluate and compare the two algorithms in their accuracy and convergence speed. We then propose a novel algorithm, combining the two, including modifications made to the WOA.

Contents

1	Introduction	1
2	Derivations and the Motivating Problem	3
2.1	Motivating Problem	3
2.2	Levenberg-Marquardt Algorithm	7
2.2.1	Implementation in R	11
2.3	Whale Optimization Algorithm	12
2.3.1	Derivations	19
2.3.2	Implementation in R	21
3	Modifications and Hybridization	22
3.1	Modifications	22
3.1.1	Lévy Flight	23
3.1.2	Chaotic Mapping and Quasirandom Number Generation	27
3.2	Hybridization	27
4	Simulation Results	29
4.1	Results	29
5	Conclusion and Future Results	39
	Acknowledgments	41
	Technology Statement	42
	Bibliography	43

Chapter 1

Introduction

Optimization plays a central role in statistical modeling, machine learning, and scientific inference. Optimization algorithms provide strategies for navigating solution spaces of all kinds, with the goal of reaching absolute optima, (and thus avoiding local optima), see Figure 1.1. From this point forward, we will understand optimization to be minimization, and thus talk about desired optima as minima. Early methods for continuous numerical optimization used gradients to find minima, a strategy which still underlies many of the most effective and popular optimization routines used today [Cauchy, 1847]. However, for more recent optimization problems which are often high dimensional and search spaces that are multimodal and noisy, the usefulness of stochastic methods is apparent.

Stochastic optimization includes elements of randomness that help the routine avoid getting stuck in local minima. The stochastic characteristic helps optimization routines solve complex problems, but often at a much higher computational expense than more traditional, deterministic methods and often with a more tenuous basis in mathematical theory.

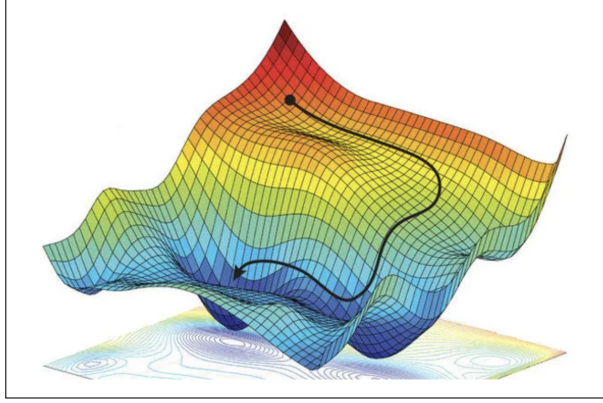


Figure 1.1: Optimization in 3 dimensions [Mehboob, 2024]

This thesis explores two optimization routines: the Levenberg–Marquardt algorithm (LMA) and the Whale Optimization Algorithm (WOA). The former is a gradient-based, non-linear least squares method that incorporates both gradient descent and Gauss-Newton optimization, while the latter is a metaheuristic algorithm inspired by the bubble-net feeding behavior of humpback whales. A comparison and hybridization of these two approaches bridges traditional deterministic optimization techniques with more recent evolutionary, stochastic methods.

This thesis is motivated by the challenges of fitting sigmoidal models to time-course data in the R package **sicegar**, which classifies and fits curves to biological data. While LMA is the current method of optimization used in **sicegar**, this work explores how a population-based stochastic algorithm, namely WOA—along with its modifications can improve robustness or convergence in the problems that **sicegar** is built to solve.

The remainder of this thesis will be structured as follows. Chapter 2 outlines the origins and structures of both algorithms, as well as details of the motivating problem. Chapter 3 explores modifications made to the algorithms, as well as their hybridization. Chapter 4 describes the results of the simulations, and Chapter 5 provides a conclusion and future directions.

Chapter 2

Derivations and the Motivating Problem

2.1 Motivating Problem

The R package `sicegar` is designed to classify and fit sigmoidal and double-sigmoidal curves to time-intensity data, providing biologically interpretable models and the related parameter estimates. Caglar and Wilke [2021] state, “This method was first developed in the context of single-cell viral growth analysis (for details, see Caglar et al. [2018]), and the package name stands for ‘SIngle CELL Growth Analysis in R.’” At its core, `sicegar` determines whether a given trajectory exhibits monotonic increasing sigmoidal behavior, rise-and-fall (double-sigmoidal) behavior, or no meaningful structured signal. Once classified, the package extracts key parameters such as midpoint times, slopes, and asymptotes, that can be directly interpreted in biological contexts.

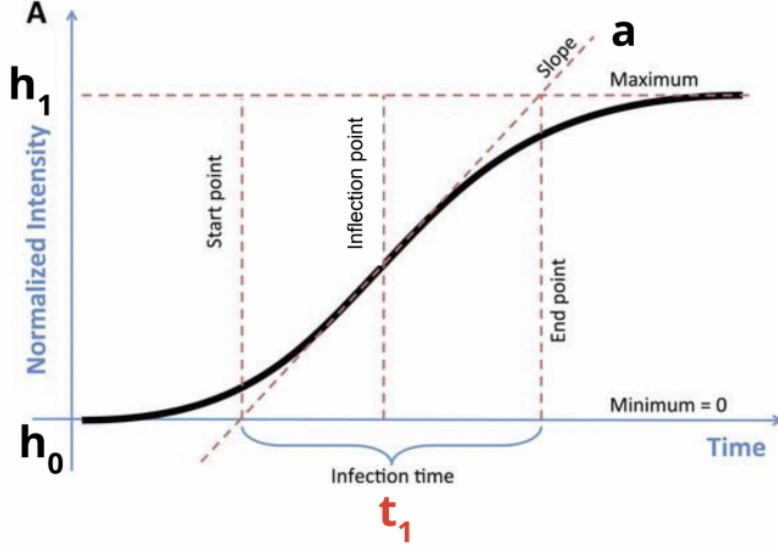


Figure 2.1: Sigmoidal curve [Caglar et al., 2018].

For sigmoidal trajectories, **sicegar** fits a logistic curve parameterized as

$$f_{sig}(x) = h_0 + \frac{h_1 - h_0}{1 + e^{-a_1(x-t_1)}},$$

where h_0 represents the minimum intensity, h_1 represents the maximum intensity, a_1 controls the steepness of the transition, and t_1 is the time at which intensity reaches half of its maximum value. The accurate estimation of the parameters in the model both ensure a good fit to the data, and allow for usable interpretation by researchers.

For trajectories exhibiting both growth and decay, **sicegar** employs a double-sigmoidal model that effectively combines two logistic transitions: one describing the increasing phase and another describing the declining phase. This model introduces additional parameters, including separate slope and midpoint terms for both the ascending and descending components, as well as a parameter controlling the final asymptotic intensity. The double-sigmoidal formulation allows the model to accurately model biologic phenomena that do not reach stasis at a maximum but fall to a lower final intensity (see Figure 2.2). The increased number of parameters in the double-sigmoidal model render a more difficult optimization problem due to the increased dimension of the optimization surface.

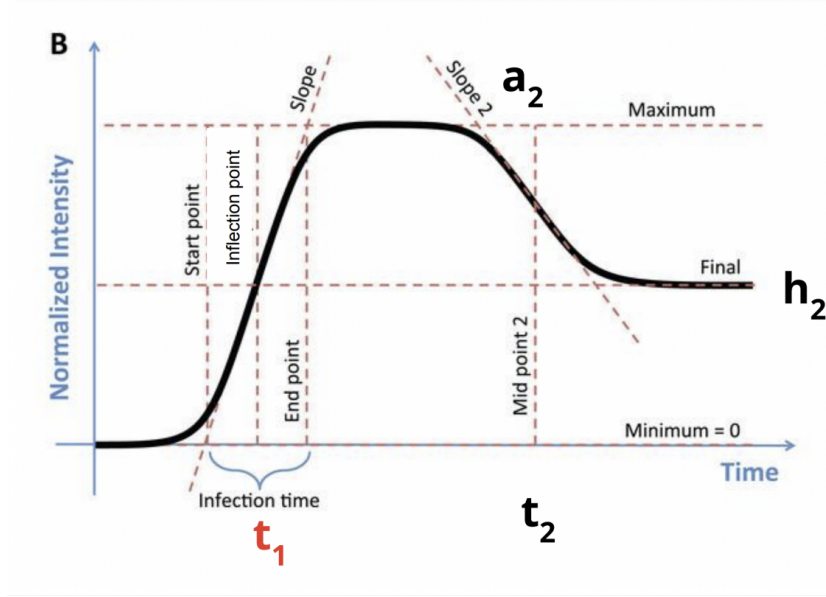


Figure 2.2: Double-sigmoidal curve [Caglar et al., 2018].

The double-sigmoidal model is defined using a piecewise formula, to avoid unfavorable edge cases that are not actually double-sigmoidal which can appear in the naïve characterization of the double-sigmoid as a product of two sigmoid models. The parameterization of the double-sigmoidal model is given here as in Caglar et al. [2018]:

Define

$$f_{base}(x) = \frac{1}{1 + e^{-a_1(x-t_1)}} \cdot \frac{1}{1 + e^{a_2(x-t_2)}}$$

then

$$I(x) = \begin{cases} c_1 f_{base}(x) + h_0, & x \leq x^*, \\ c_2 f_{base}(x) + h_2, & x > x^* \end{cases}$$

where

$$x^* = \operatorname{argmax} f_{base}(x)$$

and

$$c_1 = \frac{(h_1 - h_0)}{f_{max}}$$

$$c_2 = \frac{(h_1 - h_2)}{f_{max}}$$

with

$$f_{max} = f_{base}(x^*)$$

where, as seen in Figure 2.2, h_0 is the lower asymptote, h_1 is the upper asymptote, h_2 is the final asymptote, t_1 is the x-value corresponding to the y-value at the midpoint between h_0 and h_1 , t_2 is the x-value corresponding to the y-value at the midpoint between h_1 and h_2 , a_1 is the slope parameter for the increasing side of the double-sigmoidal curve, a_2 is the slope parameter for the decaying side of the double-sigmoidal curve, and x^* is the x-value at which the piecewise function switches from the increasing sigmoid to the decaying sigmoid. x^* is calculated numerically using R’s `optimize()` [R Core Team, 2025] function.

To estimate parameters, **sicegar** first rescales both time and intensity values to the unit interval $[0, 1]$, so that the bounds set on each parameter are not dependent on the scale of the actual data [Caglar et al., 2018]. Parameter estimation is then performed via non-linear least squares using the Levenberg–Marquardt algorithm implemented through the `minpack.lm` [Elzhov et al., 2023] package in R. The objective function that is being minimized is the residual sum of squares between observed intensities and model predictions. To avoid issues of local minima and poor initialization, the Levenberg–Marquardt algorithm is run from multiple randomized initial parameter guesses constrained within the user-defined bounds.

After fitting both sigmoidal and double-sigmoidal models to a dataset, **sicegar** uses AIC to pick the more accurate curve, or if neither meet a pre-specified threshold, the fit is labeled as “ambiguous” and no curve is fit to the data [Caglar et al., 2018].

A recent application of **sicegar** demonstrates the biological importance of accurate parameter estimation. In a study investigating the timing of RpoS-dependent gene transcription in *E. coli* under various stress conditions, Adams et al. [2023] used **sicegar** to fit sigmoidal and double-sigmoidal models to RNA-seq time-course data and extracted midpoint times (t_1 values) as estimates of transcriptional onset time. In that study, the midpoint parameter t_1 served as a measure of when individual genes became activated in response to environmental stress. By comparing onset times across conditions such as high osmolarity, low temperature, and stationary phase entry,

the authors were able to explore how gene onset time varies depending on stress context [Adams et al., 2023].

The importance of reliable parameter estimates motivates further investigation into understanding parameter estimation for sigmoidal and, especially, double-sigmoidal models. Because these models are non-linear, multi-parameter, and high dimensional, their estimation can be sensitive to initialization, noise, and optimization strategy. Enhancing the robustness, convergence behavior, or efficiency of the estimation process—particularly for the higher-dimensional double-sigmoidal case—has the potential to improve future biological inference.

2.2 Levenberg-Marquardt Algorithm

The Levenberg-Marquardt algorithm (LMA) was developed between 1944 and 1963 by Kenneth Levenberg and Donald Marquardt. Their contributions are referenced together in the name of the algorithm, which is essentially Marquardt’s improvements to Levenberg’s earlier work [Levenberg, 1944, Marquardt, 1963]. The algorithm was created to solve non-linear least squares problems, which require iterative functions to estimate model coefficients. The goal of non-linear least squares optimization—like linear least squares—is to minimize the squared distance between the data and the model. The Levenberg-Marquardt algorithm combines optimization through the gradient descent method and the Gauss-Newton method. The former uses derivatives to update the coefficients in the objective function (sum of squared errors) in the direction of steepest minimization. The Gauss-Newton method locally approximates the objective function as linear and computes updates by solving a system analogous to the normal equations of linear least squares. The LMA uses gradient descent when the current guess of the optimal values is further away from the best solution, and transitions to the Gauss-Newton as it gets closer [Gavin, 2024]. The derivations below describe this process.

We begin by writing the objective function, the sum of the squared residuals:

$$\text{SSE}(\theta) = \sum_{i=1}^n [y_i - f(x_i, \theta)]^2$$

where we have n data points (x_i, y_i) , parameter vector θ , and the value of the objective function $f(x_i, \theta)$ corresponding to a set of parameter estimates.

Gradient Descent

LMA begins by using the gradient descent method to estimate θ . This derivation follows that of Gavin [2024], removing the weight matrix W because $SSE(\theta)$ is unweighted. The gradient of the SSE objective function with respect to the parameter θ is given by

$$\begin{aligned}\frac{\partial}{\partial \theta} SSE(\theta) &= \frac{\partial}{\partial \theta} \sum_{i=1}^n [y_i - f(x_i, \theta)]^2 \\ &= 2 \sum_{i=1}^n [y_i - f(x_i, \theta)] \cdot \frac{\partial}{\partial \theta} [y_i - f(x_i, \theta)] \\ &= -2 \sum_{i=1}^n [y_i - f(x_i, \theta)] \cdot \frac{\partial f(x_i, \theta)}{\partial \theta}\end{aligned}$$

For simpler notation, we rewrite the final gradient term as

$$J = \frac{\partial f(x, \theta)}{\partial \theta}$$

where we call J the Jacobian matrix, which has dimension $n \times p$, where $p = |\theta|$. J is the matrix of partial derivatives calculated over all data n and all parameters p . We write the (i, k) entry of J

$$J_{ik} = \frac{\partial f(x_i, \theta)}{\partial \theta_k}$$

representing the local sensitivity of the model f to variation in the k -th parameter θ_k . We then rewrite the gradient with respect to θ , generating a $p \times 1$ vector, h_{gd} , to update parameters in the direction of steepest descent by multiplying the gradient by the transpose of the Jacobian:

$$h_{gd} = \alpha J^T (y - f(x, \theta))$$

where $y - f(x, \theta)$ is the vectorized form of the residuals, and the positive scalar α determines the length of the step in the steepest-descent direction. The update vector h_{gd} is added to the current parameter vector to generate the updated position. Continued updates in this manner moves the algorithm towards the minimum of the solution space.

Gauss-Newton

The Gauss-Newton method treats non-linear least squares optimization problems as locally linear, and thus treats the coefficients as quadratic near the optimal solution. For moderate non-linear least squares problems the Gauss-Newton method converges faster than the gradient descent method [Marquardt, 1963]. We begin with the same objective function:

$$\text{SSE}(\theta) = \sum_{i=1}^n [y_i - f(x_i, \theta)]^2$$

Then, we update our parameter vector to a new estimate $(\theta + \delta)$ at each iteration, where

$$f(x_i, \theta + \delta) \approx f(x_i, \theta) + J_i \delta$$

and J_i is the vector corresponding to the i -th row of the Jacobian J . We then rewrite the objective function, substituting in the new estimate.

$$\text{SSE}(\theta) = \sum_{i=1}^n [y_i - f(x_i, \theta) - J_i \delta]^2$$

In order to minimize the updated objective function, we take the derivative with respect to δ (rather than θ).

$$\frac{\partial}{\partial \delta} \sum_{i=1}^n [y_i - f(x_i, \theta) - J_i \delta]^2 = 2 \sum_{i=1}^n -J_i [y_i - f(x_i, \theta) - J_i \delta]$$

To find the minimum of $\text{SSE}(\theta)$, we set the derivative equal to zero and solve for δ , the update to our parameter vector:

$$\begin{aligned}
0 &= 2 \sum_{i=1}^n -J_i [y_i - f(x_i, \theta) - J_i \delta] \\
0 &= -2 \sum_{i=1}^n J_i [y_i - f(x_i, \theta) - J_i \delta] \\
0 &= \sum_{i=1}^n J_i [y_i - f(x_i, \theta)] - \sum_{i=1}^n J_i^2 \delta \\
\sum_{i=1}^n J_i^2 \delta &= \sum_{i=1}^n J_i [y_i - f(x_i, \theta)]
\end{aligned}$$

Writing the summations in matrix form gives,

$$\begin{aligned}
J^T J \delta &= J^T (y - f(x, \theta)) \\
\delta &= (J^T J)^{-1} J^T (y - f(x, \theta))
\end{aligned}$$

Thus, the objective function is updated at each iteration by adding $\delta = (J^T J)^{-1} J^T (y - f(x, \theta))$ to θ .

Levenberg-Marquardt Algorithm

The Levenberg-Marquardt algorithm adaptively varies between the gradient descent method and the Gauss-Newton method depending on the algorithm's proximity to a solution. The choice of method depends on the damping coefficient λ ,

$$[J^T J + \lambda I] h_{\text{lm}} = J^T (y - f(x, \theta)) \quad (2.1)$$

where small values of λ result in Gauss-Newton updates and large values of λ result in gradient descent updates. In Equation 2.1 I is the identity matrix, and h_{lm} is the updated vector generated by solving Equation 2.1. For small values of λ , Equation 2.1 approximates the Gauss-Newton update, while for large values of λ , the update becomes proportional to the gradient descent direction. Thus, rather than switching between the two methods, the Levenberg-Marquardt algorithm produces a single update that continuously varies between these behaviors.

The damping coefficient λ is initialized at a large value to ensure that the algorithm begins by taking small steps using gradient descent. If any steps do not decrease the objective value, the damping parameter is increased, otherwise, as the objective value improves, λ is decreased so that the algorithm eventually switches from gradient descent to Gauss-Newton and converges to a minimum. This allows the algorithm to use the faster and more efficient Gauss-Newton method when it is close to a solution, and the slower but more stable gradient descent method at earlier iterations [Marquardt, 1963]. It should be noted that in Marquardt’s update to Levenberg’s work, the damping parameter λ is scaled by the diagonal of the approximate Hessian $J^T J$ for each parameter [Marquardt, 1963],

$$[J^T J + \lambda \text{diag}(J^T J)] h_{\text{lm}} = J^T (y - f(x, \theta)).$$

2.2.1 Implementation in R

The Levenberg-Marquardt algorithm is implemented using the `minpack.lm` package in R available on CRAN [Elzhov et al., 2023]. Rather than requiring the user to supply analytical derivatives, `minpack.lm` approximates the Jacobian J numerically using forward finite differences,

$$J_{ik} \approx \frac{f(x_i, \theta + \epsilon_k) - f(x_i, \theta)}{\epsilon_k} \quad (2.2)$$

where ϵ_k is a small perturbation applied to the k -th parameter. This means that for a model with p parameters, `minpack.lm` requires p additional function evaluations per iteration to construct J , which increases computational cost but eliminates the burden of deriving and implementing analytical gradients for complex models such as the double-sigmoidal function used in `sicegar`. The accuracy of the Jacobian approximation thus depends on the choice of ϵ : too large and the finite difference is a poor local approximation to the true derivative; too small and the computer won’t be able to identify a difference between values (since it only calculates to a fixed number of decimal points), leading to an error. `minpack.lm` handles this automatically using an adaptive step size, which eliminates the risk of either of the ϵ related errors. The algorithm’s convergence behavior is further controlled by three hyperparameters: `ptol` and `ftol` are tolerance thresholds that determine when the algorithm has converged, declaring convergence when the relative change in the parameter estimates or the objective function value

between iterations falls below their respective thresholds, while `maxiter` sets a hard upper limit on the number of iterations, stopping the algorithm early if convergence has not been reached.

2.3 Whale Optimization Algorithm

The Whale Optimization Algorithm, proposed by Mirjalili and Lewis [2016], is a metaheuristic algorithm inspired by the bubble-net hunting strategy of humpback whales. It was created in a broader context of metaheuristic algorithms inspired by natural phenomena, including the Particle Swarm Optimization (PSO) [Kennedy and Eberhart, 1995], which mimics the social behavior of flocking birds and schools of fish, and the Ant Colony Optimization algorithm (ACO) [Dorigo et al., 1991], which is inspired by the pheromones of ants. These stochastic algorithms have gained popularity because of their ability to avoid local optima in solving high-dimensional, multi-modal problems.

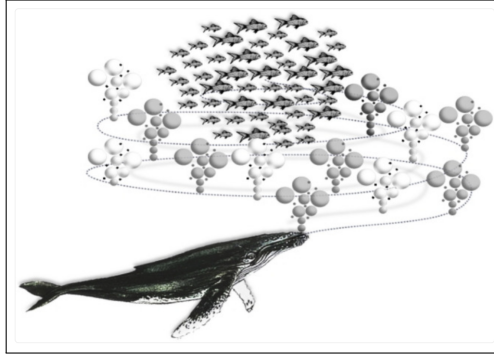


Figure 2.3: Humpback whale bubble net hunting strategy [Abdulkareem et al., 2025]

Humpback whales feed on tiny crustaceans called krill and other small schooling fish. Once a whale has located a school of these fish, it will begin to swim in a circular pattern beneath the school, releasing streams of bubbles that encircle their prey. The whale then begins to swim upwards, tightening the cylinder of bubbles, until it can swim through its bubble net, mouth open, swallowing the smaller animals [Mann et al., 2000].

The Whale Optimization Algorithm simulates this behavior by randomly distributing whales, which act as search agents, across the search space. The

position of each whale in the search space corresponds to a parameter vector, and by updating the values of said vector, we change the position of the whale. The fitness of the whale refers to the objective value corresponding to its parameter vector, that is, if we plug each of the parameter values into the objective function, the fitness is the objective value we obtain. We are interested in the parameter vector that minimizes the objective function.

WOA has two main components, which simulate the hunting strategy: Exploration and Exploitation. During Exploration, the algorithm searches the solution space broadly and randomly to identify promising regions. During Exploitation, the algorithm searches locally, using either a spiral updating equation or an encircling equation to update a given search agent's (whale's) position in the direction of the current best solution.¹

Exploration (Global Search): The algorithm updates each search agent's position toward a *randomly selected* whale $\vec{X}_{rand}(t)$, in order to search globally across the solution space, using the *encircling prey* approach.

Exploitation (Local Search): The algorithm updates each search agent's position toward the *current best solution* $\vec{X}^*(t)$, either via the *encircling prey* mechanism or the *spiral updating* position, refining the search in a promising region.

There are two approaches to updating the position of each search agent. They are as follows:

- *Encircling prey:* The first component of the algorithm simulates the encircling behavior of humpback whales once they have located a school of fish. It is used during both the Exploration and Exploitation phase to update the location of each search agent. The current position (parameter vector) of a given search agent at time t , $\vec{X}(t)$, is updated using Equations 2.3 & 2.4. If the resulting parameter vector, at time $t + 1$, $\vec{X}(t + 1)$, improves the search agent's position (i.e., moves it closer to a minimum) then the search agent's position will update to the new and better solution. The strategy is modeled by the following two equations:

¹The term "search agent" denotes the parameter vector corresponding to the position of a "whale". In this text, the two terms are synonymous.

$$\vec{X}(t+1) = \vec{X}^*(t) - \vec{A} \cdot \vec{D} \quad (\text{Exploit}) \quad (2.3)$$

$$\vec{D} = \left| \vec{C} \cdot \vec{X}^*(t) - \vec{X}(t) \right| \quad (2.4)$$

where t indicates the current iteration, $\vec{A}$ and $\vec{C}$ are coefficient vectors, $\vec{X}^*(t)$ is the position vector of the current *best* solution in the first $t-1$ iterations, and $\vec{X}(t)$ is the current position vector. The general form of Equations 2.3 and 2.4 show up in two places in the algorithm: during Exploration (global search), where the current search agent moves relative to another *randomly* selected search agent, (see Equations 2.11 & 2.12), and during Exploitation where the current agent moves relative to the current best solution (see Equations 2.3 & 2.4). We recall that Exploitation refers to the later parts of the algorithm when the global search has been completed and the algorithm restricts its search to the most promising regions of the solution space. This is mostly controlled by the decreasing influence of random coefficients in the later part of the algorithm, which prevent large movements around the solution space. Equations 2.3 and 2.4 represent the encircling prey method during Exploitation, when the search agent's position is updated in the direction of the current best solution. In Equation 2.4 the distance of the current solution from a randomly scaled version of the current best solution is calculated, and then that distance vector $\vec{D}$ is scaled by the random, but decreasing vector $\vec{A}$ and subtracted from the current best solution. If the resulting parameter vector evaluates to a lower objective value, then it will become the new $\vec{X}^*$. Equation 2.3 updates the position of the current search agent in the direction of the current best solution, while preserving some element of randomness. The coefficient vectors responsible for preserving the stochastic nature of the algorithm are $\vec{A}$ and $\vec{C}$. They are calculated as follows:

$$\vec{A} = 2\vec{a} \cdot \vec{r} - \vec{a} \quad (2.5)$$

$$\vec{C} = 2\vec{r} \quad (2.6)$$

where $\vec{r}$ is a vector of all r which is randomly generated uniformly from $[0, 1]$, and $\vec{a}$ is a vector of all a which is a scalar that decreases linearly from 2 to 0 over the course of the iterations:

$$a = 2 \left(1 - \frac{t}{\text{maxIter}} \right) \quad (2.7)$$

In Equation 2.7, t is the current iteration and maxIter is the maximum number of iterations. It should be noted that a , and r are both scalars, and $\vec{A}$, $\vec{C}$, $\vec{a}$, and $\vec{r}$ are composed of repetitions of those scalars, that is, the same scalar values are used to update all dimensions, for each whale, in each iteration.

$$\vec{a} = (a, a, \dots, a)$$

$$\vec{r} = (r, r, \dots, r)$$

$$\vec{A} = (2ar - a, 2ar - a, \dots, 2ar - a)$$

$$\vec{C} = (2r, 2r, \dots, 2r)$$

It is worth mentioning that the relative simplicity of the vectors $\vec{a}$ and $\vec{r}$, and in turn $\vec{A}$ and $\vec{C}$, lends to easy modification, and the majority of modified WOA versions involve some changes to these vectors.

- *Spiral updating position:* The second method for updating the position of a search agent happens only during Exploitation (local search). A spiral equation that mimics the helix-shaped swimming pattern of a whale when it is trapping a school of fish in a cylinder of bubbles is used to update the position of the current search agent. The equation is as follows:

$$\vec{X}(t+1) = \vec{D}' \cdot e^{bl} \cdot \cos(2\pi l) + \vec{X}^*(t) \quad (\text{Exploit}) \quad (2.8)$$

$$\vec{D}' = \left| \vec{X}^*(t) - \vec{X}(t) \right| \quad (2.9)$$

where $\vec{D}'$ is the distance between the position of the current search agent and the current best solution, b is a constant for defining the shape of the logarithmic spiral, l is a random number in $[-1, 1]$. $\vec{D}'$ is multiplied by the constant, $e^{bl} \cdot \cos(2\pi l)$, and added to the current best solution. If the resulting parameter vector evaluates to a smaller value in the objective function, then $\vec{X}(t+1)$ replaces $\vec{X}^*(t)$.

Note that humpback whales swim around their prey in a shrinking circle and along a spiral-shaped path simultaneously. As $\vec{X}(t)$ approaches $\vec{X}^*(t)$, $\vec{D}'$, and thus the whole term $\vec{D}' \cdot e^{bl} \cdot \cos(2\pi l)$ in Equation 2.8 decreases, mimicking the shrinking behavior the bubble-net hunting strategy. The algorithm assumes that there is an equal probability of choosing between either the encircling equation, Equation 2.3, or the spiral updating mechanism, Equation 2.8, to update the position of whales. This can be represented in the following model:

$$\vec{X}(t+1) = \begin{cases} \vec{X}^*(t) - \vec{A} \cdot \vec{D}, & \text{if } p < 0.5, |A| < 1 \quad (\text{Exploit}) \\ \vec{X}_{rand}(t) - \vec{A} \cdot \vec{D}, & \text{if } p < 0.5, |A| \geq 1 \quad (\text{Explore}) \\ \vec{D}' \cdot e^{bl} \cdot \cos(2\pi l) + \vec{X}^*(t), & \text{if } p \geq 0.5 \quad (\text{Exploit}) \end{cases} \quad (2.10)$$

where $p \sim \text{Unif}(0, 1)$. When $p < 0.5$, the algorithm proceeds to check whether $|A| \geq 1$. $|A|$ is more likely to be larger than one at earlier iterations since the value of a is smaller in later iterations.² When $|A| \geq 1$, the algorithm updates the position of the current search agent in the direction of another random agent using the following equation:

$$\vec{X}(t+1) = \vec{X}_{rand}(t) - \vec{A} \cdot \vec{D} \quad (\text{Explore}) \quad (2.11)$$

$$\vec{D} = \left| \vec{C} \cdot \vec{X}_{rand}(t) - \vec{X}(t) \right| \quad (2.12)$$

Equation 2.11 is a slight variation of Equation 2.3 where instead of the current position being updated towards the current best, it is updated in the direction another *randomly* selected search agent $\vec{X}_{rand}$. The algorithm uses

²Note to reader: if you want further convincing of this fact, refer to Equation 2.5 and try plugging in values of a that are close to 2.

this random update during Exploration, which we recall defines the phase of the algorithm that is responsible for the global search of the solution space. A desirable characteristic of Exploration is sufficient searching across the whole solution space, including random jumps that ensure the algorithm does not get stuck in any local minima. In Equation 2.12, the distance between the current search agent and another random search agent is calculated. The parameter vector of the randomly selected search agent is multiplied by $\vec{C}$, to inject randomness into the vector. The updated position of the current whale is then calculated by subtracting a scaled version of the distance vector $\vec{D}$ from the randomly chosen whale. If the fitness of $\vec{X}(t+1)$ is better than $\vec{X}(t)$, then the position of the whale is updated to $\vec{X}(t+1)$. Better fitness in our case means the corresponding parameter vector yields a lower objective value. The random movement of search agents is the basis for the Exploration phase of the algorithm, allowing the whales to search the solution space broadly and stochastically.

A pseudo code structure for the algorithm is given in Figure 2.3.

Algorithm 1 Whale Optimization Algorithm (WOA) [Mirjalili and Lewis, 2016]

```

Initialize the whales population  $\vec{X}_i$  ( $i = 1, 2, \dots, n$ )
Calculate the fitness of each search agent
 $\vec{X}^*$  = the best search agent
while  $t < \text{maximum number of iterations}$  do
    for each search agent do
        Update  $a$ ,  $A$ ,  $C$ ,  $l$ , and  $p$ 
        if  $p < 0.5$  then
            if  $|A| < 1$  then
                Exploit: Update the position of the current search agent by
Eq. 2.3
            else if  $|A| \geq 1$  then
                Explore: Select a random search agent ( $\vec{X}_{\text{rand}}$ )
                Update the position of the current search agent by Eq. 2.11
            end if
        else if  $p \geq 0.5$  then
            Exploit: Update the position of the current search agent by
Eq. 2.8
        end if
    end for
    Check if any search agent goes beyond the search space and project it
    back onto the feasible region.3
    Calculate the fitness of each search agent
    Update  $\vec{X}^*$  if there is a better solution
     $t = t + 1$ 
end while
return  $\vec{X}^*$ 

```

An important checkpoint in the algorithm verifies whether $|A| \geq 1$. We note that when a is large (close to 2), it is often the case that $|A| \geq 1$. When $|A| \geq 1$, the algorithm explores (searches globally) and when $|A| < 1$ the algorithm exploits (searches locally).

³If a search agent exceeds the bounds in any dimension, its coordinate in that dimension is set equal to the boundary value.

2.3.1 Derivations

It is essential to understand the Whale Optimization Algorithm's balance between Exploration and Exploitation, so as to understand whether the algorithm converges prematurely (i.e., the algorithm switches from exploring to exploiting too early). In order to understand this, some information about $\vec{A}$ will be necessary.

1. First, find $\mathbb{E}(|A|)$:

$$\begin{aligned}\mathbb{E}(|A|) &= \int_{-a}^a |x| \cdot \frac{1}{2a} dx \\ &= \frac{1}{2} \int_{-a}^a |x| \cdot \frac{1}{a} dx \\ &= \frac{1}{a} \int_0^a x dx \\ &= \frac{1}{a} \left(\frac{1}{2} x^2 \right) \Big|_0^a \\ &= \frac{1}{a} \cdot \frac{a^2}{2} = \frac{a}{2}\end{aligned}$$

2. Then we find $P(|A| < 1)$ to understand when the algorithm will switch from exploring to exploiting.

Recall: $\vec{A} = 2\vec{a} \cdot \vec{r} - \vec{a}$

$$\begin{aligned}\mathbb{P}(|A| < 1) &= \mathbb{P}(|2ar - a| < 1) \\ &= \mathbb{P}(|a(2r - 1)| < 1) \\ &= \mathbb{P}\left(|2r - 1| < \frac{1}{a}\right)\end{aligned}$$

We then have two cases:

- **Case 1:** $a \leq 1$

Then $\frac{1}{a} \geq 1$ and so we have:

$$|2r - 1| \leq 1 \leq \frac{1}{a},$$

Then, it follows by transitivity

$$|2r - 1| \leq \frac{1}{a}$$

And since r is a continuous random variable,

$$\mathbb{P}\left(|2r - 1| = \frac{1}{a}\right) = 0$$

So we have:

$$\mathbb{P}\left(|2r - 1| < \frac{1}{a}\right) = 1$$

Or equivalently:

$$\mathbb{P}(|A| < 1 \mid a \leq 1) = 1$$

• **Case 2:** $a > 1$

We start with

$$|2r - 1| < \frac{1}{a}$$

So we can write

$$-\frac{1}{a} < 2r - 1 < \frac{1}{a}$$

Adding 1 and dividing by 2 gives

$$\frac{1 - \frac{1}{a}}{2} < r < \frac{1 + \frac{1}{a}}{2}$$

So the condition $|A| < 1$ is equivalent to r falling between these values. Because r has a uniform distribution, the probability that r is between these values is equivalent to the width of the interval, or their difference.

$$\frac{1 + \frac{1}{a}}{2} - \frac{1 - \frac{1}{a}}{2} = \frac{\frac{2}{a}}{2} = \frac{1}{a}$$

So, since we've shown that $\mathbb{P}\left(r \in \left(\frac{1 - \frac{1}{a}}{2}, \frac{1 + \frac{1}{a}}{2}\right)\right) = \mathbb{P}(|A| < 1 \mid a > 1)$, we have,

$$\mathbb{P}(|A| < 1 \mid a > 1) = \frac{1}{a}$$

So, in conclusion we have

$$\mathbb{P}(|A| < 1) = \begin{cases} 1, & \text{if } a \leq 1, \\ \frac{1}{a}, & \text{if } a > 1. \end{cases}$$

We recall that the value $|A|$ determines whether the algorithm explores (searches globally) or exploits (searches locally). When $|A| < 1$, the algorithm exploits. It is important to have a sense of when this happens, because if WOA begins Exploitation prematurely without sufficiently searching the whole solution space, we risk converging to a local minimum, since all possible regions may not have been explored yet. We recall that a decreases linearly from 2 to 0 over the course of all iterations. Therefore in the second half of the algorithm’s search, the probability of exploiting is 1. In the first half of the algorithm’s search, the probability of exploiting is $\frac{1}{a}$, which means at least $\frac{1}{2}$. Thus, the probability that the algorithm conducts a global search through Exploration is at most $\frac{1}{2}$. WOA has been shown to converge prematurely, [Nadimi-Shahraki et al., 2023] which makes sense given the relatively high probability that the algorithm chooses to search locally (exploit), even at early stages of the optimization routine.

2.3.2 Implementation in R

The Whale Optimization Algorithm is implemented using the R package `metaheuristicOpt`, which provides a unified framework for population-based metaheuristic optimization methods [Riza et al., 2024]. The package includes implementations of several nature-inspired algorithms, including WOA, and allows the user to specify objective functions, parameter bounds, population size, number of iterations, and other algorithm-specific control parameters within a consistent interface. The implementation of the algorithm follows closely from the pseudo-code outlined in Figure 2.3.

Chapter 3

Modifications and Hybridization

3.1 Modifications

Metaheuristic algorithms are stochastic optimization routines designed for complex optimization problems. They were created to thrive in problems that were poorly understood, or poorly optimized by traditional methods, often drawing inspiration from natural phenomena such as biological evolution, swarm behavior, and physical annealing. Among metaheuristic algorithms, WOA is an appealing choice for this thesis because of its simplicity and small number of hyperparameters. These characteristics lend the algorithm well to modification. Modified versions of WOA incorporating chaotic maps [Nadimi-Shahraki et al., 2023], Lévy flights [Ling et al., 2017], and hybridization with deterministic routines [Nadimi-Shahraki et al., 2023] have shown improved performance on benchmark optimization problems. Investigating whether these enhancements also improve real biological model fitting is both methodologically interesting and practically valuable.

Nadimi-Shahraki et al. [2023] showed that WOA, in its unmodified form, faces a multitude of shortcomings. First, they show that WOA converges prematurely, i.e., it does not maintain a sufficient balance between global searching (exploring) and local searching (exploiting). Nadimi-Shahraki et al. [2023] ran WOA on a series of test problems with known optima, including unimodal, multimodal, as well as hybrid and composition test functions which are more complex. They ran WOA on these functions and compared

the algorithmically estimated optima to the true values. They found for uni-modal test problems that “WOA cannot properly approximate or exploit the promising area” and that for multimodal tests that “whales cannot efficiently explore the entire solution space and locate the optimal solution.” The primary issue they identify is that in converging prematurely, WOA does not sufficiently explore the entire search space, and converts to the exploiting mechanism before the most promising region has been accurately identified.

Many other researchers identified similar issues, [Ling et al., 2017], [Ding et al., 2020], and the following subsections detail possible solutions.

3.1.1 Lévy Flight

One solution to the premature convergence of the Whale Optimization Algorithm is to add an additional stochastic element to the optimization routine in the form of steps drawn from a Lévy distribution. During the calculation of a search agent’s updated position, a scalar is drawn from a heavy-tailed distribution called a Lévy distribution. The distance vector (as defined in Equations 2.4, 2.9, and 2.12) is multiplied by said scalar and then the updated position is calculated. The parameter vector is then plugged into the objective function and the objective value is calculated. In most cases, the Lévy scalar is close to zero, and so the distance vector is made to be small, leading to small step sizes, however the scalar is occasionally very large, leading to large step sizes, even in the later parts of the optimization routine when the algorithm is searching locally (exploiting). Taking Lévy steps helps combat premature convergence that results from searching locally before the whole solution space has been sufficiently explored. The Lévy steps allow for a search agent to take a big step even when exploitation is happening. Ideally this could help the algorithm leap out of local minima that it has converged to prematurely.

Lévy α -stable distributions form a family of probability distributions first described by Paul Lévy in the 1920’s [Nolan, 2020]. We say that a random variable X is stable if for any $a, b > 0$, $\exists c > 0$ and $d \in \mathbb{R}$ such that

$$aX_1 + bX_2 \stackrel{d}{=} cX + d, \quad (3.1)$$

where X_1, X_2 are sampled i.i.d. from the same distribution that generated X .

A notable property of Lévy stable distributions is the existence of non-finite moments at most values of α , (see Equation 3.2). That means that instead of characterizing the distribution by means of a PDF or CDF, we must use the characteristic function. We can apply the characteristic function to both sides of Equation 3.1 to get:

$$\varphi_X(at) \cdot \varphi_X(bt) = \varphi_X(ct) \cdot e^{idt}$$

And it turns out that the only functions φ_X that satisfy the constraint that the stability property imposes are of the form $e^{-c|t|^\alpha}$, and so we have:

$$\begin{aligned} \varphi_X(t) &= \mathbb{E}[e^{itX}] = e^{-c|t|^\alpha} \\ 0 < \alpha &\leq 2, \quad c > 0. \end{aligned} \tag{3.2}$$

Functions of the form $\varphi(t) = \exp(-c|t|^\alpha)$ define valid characteristic functions if and only if $0 < \alpha \leq 2$ [Zolotarev, 1986].

Every random variable can be uniquely defined by its characteristic function [Lévy, 1925]. Conveniently, certain values of α in Equation 3.2 generate the distributions of some familiar random variables.

The characteristic function of a random variable $Z \sim N(0, \sigma^2)$ is

$$\varphi_Z(t) = e^{-\frac{\sigma^2}{2}t^2},$$

Taking the characteristic function given in Equation 3.2 and setting $\alpha = 2$ we get

$$\varphi_Z(t) = e^{-ct^2},$$

then letting $c = \frac{\sigma^2}{2}$ we see that Z has a Lévy α -stable distribution with $\alpha = 2$.

We can also show that setting $\alpha = 1$ gives a Cauchy distribution. A standard (symmetric) Cauchy random variable with scale parameter $\gamma > 0$, W , has characteristic function

$$\varphi_W(t) = e^{-\gamma|t|},$$

Setting $\alpha = 1$ in Equation 3.2 we get

$$\varphi_W(t) = e^{-c|t|},$$

then letting $c = \gamma$, we see that W has a Lévy α -stable distribution with $\alpha = 1$.

Within the broader family, the name “Lévy distribution” sometimes refers specifically to the positive $\alpha = \frac{1}{2}$ stable distribution with a heavy right tail. Figure 3.1 shows the Lévy distribution, as well as a Normal and Cauchy distribution which occur when $\alpha = 2$ and $\alpha = 1$ respectively.

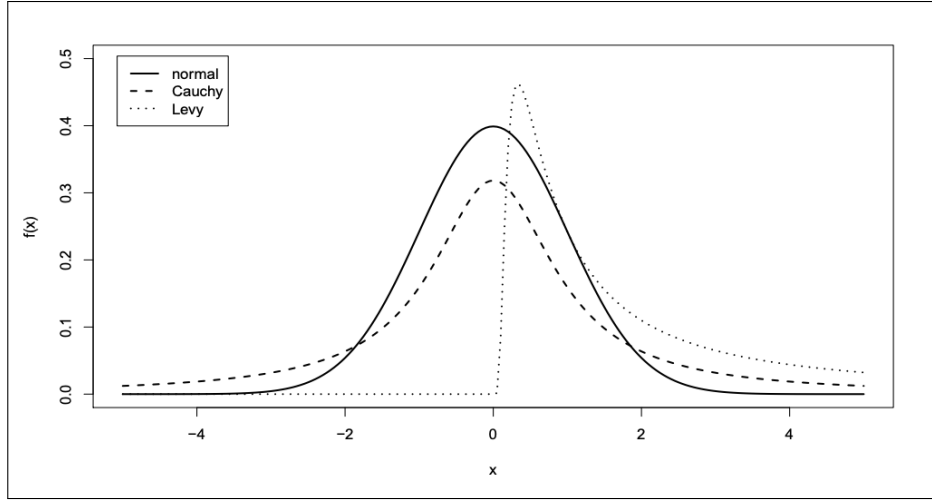


Figure 3.1: Graphs of standardized normal $N(0, 1)$, Cauchy(1,0) and Lévy(1,0) densities [Nolan, 2020]

Since the probability densities of Lévy stable random variables, (except for a few special cases such as the Gaussian and Cauchy distributions) do not admit closed-form expressions, we must characterize them through alternative mathematical tools—most importantly, characteristic functions.

In the Lévy variant of WOA, step sizes are drawn randomly from a Lévy distribution to move the search agents around the search space. The heavy-tailed characterization of Lévy distributions means that the majority of step sizes drawn from distribution are small, but occasionally large steps are drawn ensuring that search agents are able to jump out of local minimums, and successfully conduct a global search of the solution space.

Step sizes are drawn using Mantegna’s Algorithm which approximates a Lévy distribution [Mantegna, 1994]. The algorithm for step sizes s , (random deviates), is

$$s = \frac{u}{|v|^{1/\alpha}}, \quad (3.3)$$

where

$$u \sim \mathcal{N}(0, \sigma_u^2), \quad v \sim \mathcal{N}(0, 1),$$

and the scale σ_u is chosen to match the appropriate Lévy tail:

$$\sigma_u = \left(\frac{\Gamma(1 + \alpha) \sin(\frac{\pi\alpha}{2})}{\Gamma(\frac{1+\alpha}{2}) \alpha 2^{(\alpha-1)/2}} \right)^{1/\alpha}. \quad (3.4)$$

In the Lévy variant of WOA, α in Equations 3.3, and 3.4 is set equal to 1.5.

Mantegna's algorithm does not perfectly approximate a Lévy distribution, though for the purpose of drawing step sizes from a heavy-tailed Lévy-approximate distribution, the algorithm works well. Figure 3.2 shows both Mantegna's algorithm and the Lévy distribution with parameter $\alpha = 1.5$.

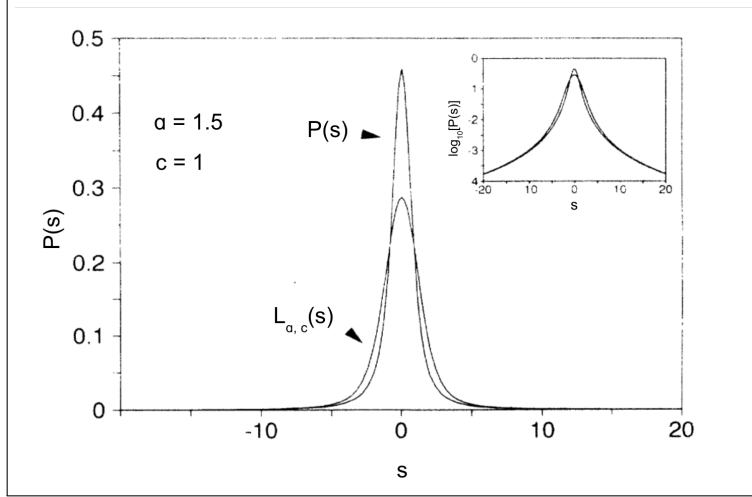


Figure 3.2: Density of Mantegna's algorithm $P(s)$ compared to Lévy distribution $L_{1.5,1}(s)$ [Mantegna, 1994]. α is the stability index and c is a scale parameter, the same as the α and c in Equation 3.2. In this picture, c is analogous to the standard deviation in a Gaussian distribution, controlling the spread.

Mantegna notes that for $|s| > 10$ the functions are almost coincident.

3.1.2 Chaotic Mapping and Quasirandom Number Generation

Two additional variants to the WOA were each tested, although their respective benefits do not completely align with the geometry of our surface of interest. Both chaotic mapping and quasirandom number generation were implemented in an effort to improve the performance of the algorithm, though neither improved performance in the context of this problem. The use of chaotic mapping in place of traditional pseudo-random number generators has been shown to improve performance when applied to the “generation of the initial [whale] population, producing values in control parameters, and updating the position of the search agents.” [Nadimi-Shahraki et al., 2023] Chaotic mapping ensures a more even distribution of random values than the pseudorandom number generators built into R.

The use of quasirandom number generation via Sobol sequences as a replacement for traditional pseudorandom number generators in the WOA does not appear to be documented in the existing literature. However, the motivation for this modification is conceptually similar to the use of chaotic maps in metaheuristic optimization: to promote a more uniformly distributed sampling of the search space during initialization. Because Sobol sequences are low-discrepancy sequences, they reduce clustering and gaps that may arise under pseudorandom sampling, thereby improving coverage of the solution space and potentially enhancing global search performance.

3.2 Hybridization

The proposed hybrid algorithm combines the global search capability of WOA with the precise local convergence of LMA. WOA is run for the majority of its iterations, before LMA is initialized from the 10 best WOA candidates, i.e., the coordinate vectors corresponding to the 10 lowest objective values found up to that point. The transition from WOA to LMA is governed by a population diversity metric computed at each iteration. Following Nadimi-Shahraki et al. [2023], diversity is defined as

$$\text{Diversity}(t) = \frac{1}{D} \sum_{d=1}^D \frac{1}{N} \sum_{i=1}^N |\text{median}\{x_d(t)\} - x_{id}(t)|$$

where N is the number of whales, D is the number of dimensions, $x_{id}(t)$ is the

position of the i -th whale in dimension d at iteration t , and $\text{median}\{x_d(t)\}$ is the median position across all whales in dimension d . Intuitively, diversity measures how spread out the population of search agents is across the search space — a high value indicates global exploration, while a low value indicates that the population has converged to a local region of the search space.

The transition from WOA to LMA is triggered when diversity falls below a certain percentage of its initial value $\text{Diversity}(0)$ (default is 5%), indicating that the population has sufficiently converged and WOA has likely identified the region where the objective function is optimal.

Upon triggering, LMA is initialized from the k best whale positions, where k is a user-specified hyperparameter (default capped at 10). To ensure that the k starting points are not redundant, a minimum distance filter is applied: a candidate whale is only retained as an LMA seed if its Euclidean distance from all already-selected seeds exceeds $d_{\min}$, a fraction of the search space diameter. Here we use $d_{\min} = 0.01$, meaning each seed must be at least 1% of the search space diameter from all others, though the optimal value of this hyperparameter has not yet been formally evaluated. LMA is run independently from each seed that passes the minimum distance threshold, and the solution with the lowest SSE across all runs is returned as the final estimate.

The diversity percentage that triggers the change from WOA to LMA can be set by the user, along with the minimum distance argument $d_{\min}$, the maximum number of iterations that WOA can run, and k the number of seeds (WOA candidates) that LMA will use as starting points. The preexisting hyperparameters of both functions also still exist, i.e., number of search agents for WOA and `ftol` and `ptol` which control convergence in LMA.

It should be noted that alterations to these hyperparameters greatly impact both the computation time and accuracy of the algorithm. LMA is a much faster algorithm than WOA, and thus higher diversity thresholds greatly reduce computation time since the hybrid algorithm spends little time running WOA. High diversity thresholds also ensure that LMA runs from variable seeds since WOA won't have been able to converge to a single minimum. If WOA runs too long, then LMA will run from only one candidate seed since WOA will have all k of its best solutions in close proximity.

Chapter 4

Simulation Results

4.1 Results

The initial goal of this thesis was to improve the optimization of double-sigmoidal curves to fit time-course data. At the outset of project, the geometry of the double-sigmoidal space was unknown, and given its high dimension and complicated generating function, the Whale Optimization Algorithm seemed like an appropriate candidate for improving the estimation process. However, after many simulations of WOA — and its variants, including chaotic mapping, quasirandom number generation, and Lévy flight variations — it was clear that the existing method of the Levenberg-Marquardt Algorithm is in fact the most efficient and most accurate solution to the problem.

Researchers Transtrum et al. [2011] describe non-linear least squares optimization problems similar to the double-sigmoid as having a solution space characterized by a narrow canyon or valley, which is well suited for the gradient-based LMA. In an attempt to verify this characteristic of our problem, we performed a grid search across the solution space.

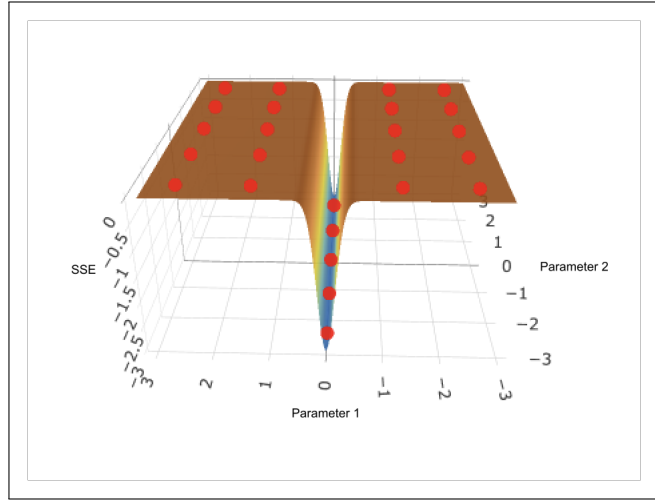


Figure 4.1: Narrow canyon in 2 dimensions with parameter grid overlaid

Performing a grid search across the optimization surface allows us to pull snapshots of the SSE value at different parameter combinations, even without being able to visualize the surface itself. If the optimization surface is characterized by a narrow canyon, then we expect a handful of parameter combinations to correspond to small SSE values at the bottom of the canyon, and the majority of SSE values to be high, on the plateaus above the canyon as can be seen in Figure 4.1. We don't expect to see intermediate values since the probability of our parameter grid intersecting over the steep walls of the valley is highly unlikely. Each dimension (parameter) was sequenced by 7, and the endpoints were removed, leaving a $5 \times 5 \times 5 \times \dots$ grid across the solution space. 5 values for each of 7 dimensions generated $5^7 = 78,125$ parameter combinations.

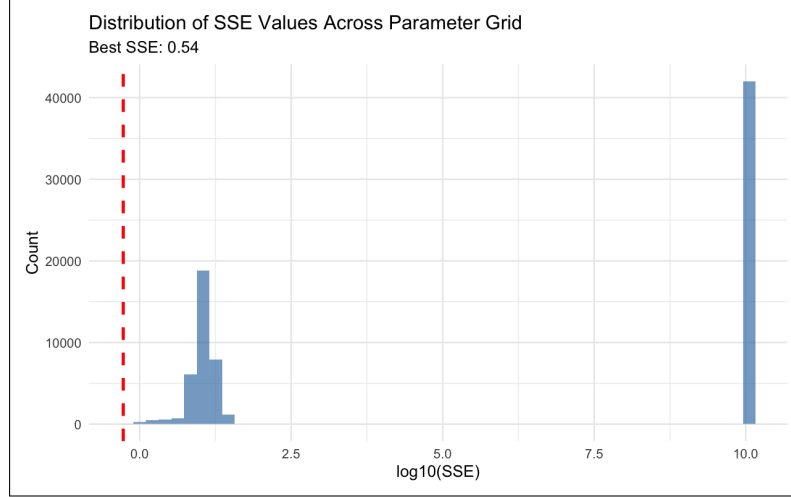


Figure 4.2: SSE values generated by parameter grid over double-sigmoidal solution space²

The results in Figure 4.2 affirm what we expected to see: some parameter combinations with small SSE values (at the bottom of the canyon), none at intermediate values, and many at high SSE values (plateaus above the canyon). Though this does not provide definitive proof that the solution space matches the geometry described by Transtrum et al. [2011], it does provide reassurance to that fact, and affirms that LMA is an effective choice for the double-sigmoidal surface.

Algorithm	Mean Obj.	Median Obj.	Min Obj.	Max Obj.	Failed	Mean Acc.	Median Acc.	Min Acc.	Max Acc.
LMA with random starts	1.74×10^{-2}	1.70×10^{-2}	8.89×10^{-3}	2.94×10^{-2}	0	1.79	1.74	0.164	6.85
WOA	5.92×10^{-1}	6.15×10^{-1}	1.68×10^{-2}	1.39	0	98.4	101.5	1.24	239
WOA Lévy	4.17×10^{-1}	5.05×10^{-1}	1.16×10^{-2}	1.19	0	105	109	2.08	239
WOA-LMA Hybrid	2.33×10^{-1}	1.38×10^{-2}	8.94×10^{-3}	9.61×10^{-1}	26	—	1.69	0.0331	7.31×10^6

Table 4.1: Summary of algorithm performance on the double-sigmoidal problem across 200 seeds. Obj. refers to the SSE objective value; Acc. refers to the Euclidean distance from the true parameters in normalized space. The WOA-LMA Hybrid mean accuracy is suppressed (—) as 26 failed runs produce extreme outliers; the median is reported instead.

On the double-sigmoidal problem, LMA was run with repeated random starts. Table 4.1 provides the results of running four different optimization algorithms on the double-sigmoidal problem. The best solution was returned

²The “Best SSE: 0.54” corresponds to the lowest SSE found for the particular double-sigmoidal dataset that the parameter grid was tested on.

after either 20 successful fits, or 500 attempts, whichever came first. LMA with random starts demonstrates clear superiority in both the objective value and the accuracy of the parameter estimates. It achieves a mean SSE of 1.74×10^{-2} and a mean Euclidean distance from the true parameters of 1.79, with no failed runs across all 200 seeds. The standalone WOA and WOA Lévy algorithms perform considerably worse on this problem, with mean SSEs of 5.92×10^{-1} and 4.17×10^{-1} respectively, and mean parameter distances exceeding 98 and 105, indicating that both might be converging to regions of the parameter space with plausible objective values but incorrect parameter estimates, likely due to the challenges posed by the scale of the slope parameters a_1 and a_2 , which differ from the other parameters. The WOA-LMA Hybrid failed 26 of 200 runs entirely, producing an extremely large mean parameter distance of 4.27×10^4 because of outliers reaching 7.31×10^6 . The median SSE of 1.38×10^{-2} and median parameter distance of 1.69 are competitive with LMA, suggesting that when the initial WOA phase of the hybrid algorithm identifies the correct neighborhood, LMA is able to converge successfully. This failure rate likely reflects cases where the WOA phase does not achieve sufficient diversity reduction to trigger the LMA step, leaving the algorithm with a poor WOA solution. Together, these results suggest that the LMA is the most effective algorithmic solution to the double-sigmoidal optimization problem, both in accuracy and computation time: LMA with random starts is roughly 10 times faster than the next best candidate.

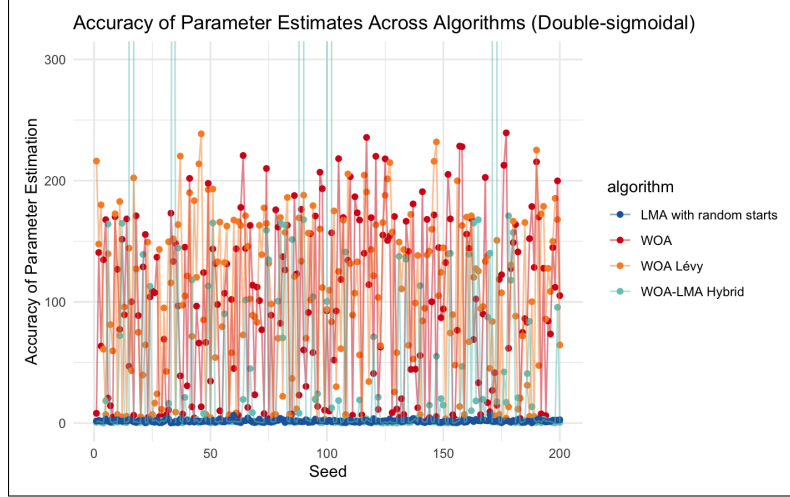


Figure 4.3: Accuracy of parameter estimates over 200 double-sigmoidal datasets

Figure 4.3 shows that the WOA-LMA Hybrid produces a small number of extreme outliers which distort the scale of the plot. The y-axis is thus capped at 300 for visual clarity; excluding these failed runs, the hybrid’s parameter estimates are broadly comparable to the other WOA based algorithms, which are all noticeably less accurate than LMA with random starts.

Once we had concluded that LMA is the correct choice for the optimization of double-sigmoidal least squares problems, we pivoted to exploring multimodal objective surfaces that would be better suited for the hybrid algorithm defined in Section 3.2. The idea behind this pivot is that the hybrid algorithm improves upon the existing WOA method by decreasing the computation time, and thus choosing an objective function that is well suited to WOA will allow us to see the improvements of the hybrid algorithm most clearly.

The Rastrigin function is a commonly used benchmark function for optimization algorithms due to the multimodal geometry of its surface (see Figure 4.4) [Rastrigin, 1974, Chen et al., 2025]. The dimension of the Rastrigin function can be increased to amplify the difficulty of the problem. The function is

$$f(\theta) = Ad + \sum_{i=1}^d (\theta_i^2 - A \cos(2\pi\theta_i))$$

where $A = 10$ to match convention. In this thesis the surface was optimized in $d = 5$ dimensions:

$$f(\theta_1, \theta_2, \theta_3, \theta_4, \theta_5) = 50 + \sum_{i=1}^5 (\theta_i^2 - 10 \cos(2\pi\theta_i))$$

The global minimum of the surface is where $f(\theta) = 0$ and $\theta_i = 0$. The bounds of the solution space are $[-5.12, 5.12]$ in all dimensions.

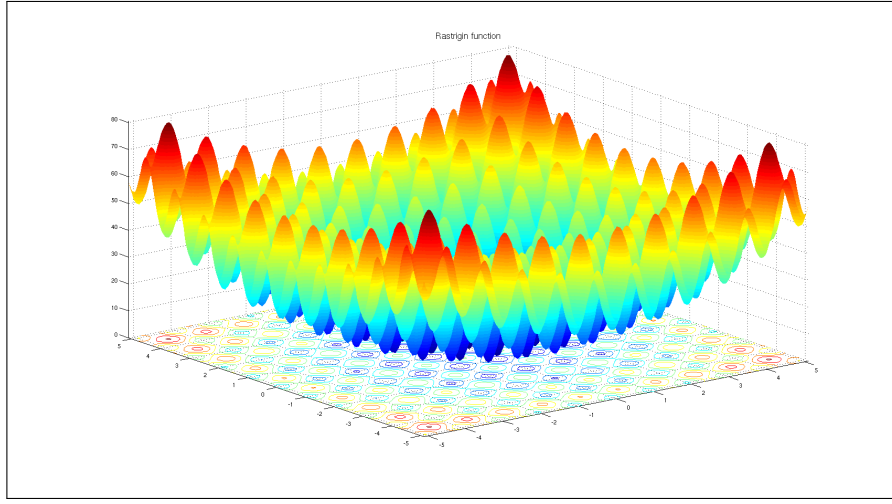


Figure 4.4: Rastrigin function in 2 dimensions
[Diegotorquemada, 2010].

An important note for the application of the hybrid algorithm to functions like the Rastrigin, is that it must be possible to write the objective function in a form that mimics the equation of SSE. The Levenberg-Marquardt Algorithm is specifically designed to solve nonlinear least squares problems, meaning it requires the objective function to take the form of a sum of squared residuals:

$$\text{SSE}(\theta) = \sum_{i=1}^d r_i(\theta)^2$$

where each $r_i(\theta)$ is a scalar residual. This structure is essential because LMA constructs its parameter update using the Jacobian matrix J , where entry $J_{ik} = \frac{\partial r_i(\theta)}{\partial \theta_k}$ captures the sensitivity of the i -th residual to the k -th parameter.

The update step relies on the approximation $J^T J \approx \nabla^2 \text{SSE}(\theta)$, which is only valid when the objective is a sum of squares. If the objective function cannot be written in this form, the Jacobian loses its meaning and LMA cannot compute a meaningful update direction.

The Rastrigin function is not naturally a sum of squares — it is a sum of per-dimension terms:

$$f(\theta) = 10d + \sum_{i=1}^d (\theta_i^2 - 10 \cos(2\pi\theta_i))$$

However, we can observe that each term in the sum is non-negative, since $\theta_i^2 - 10 \cos(2\pi\theta_i) + 10 \geq 0$ for all θ_i . This allows us to rewrite the function as a sum of squares by defining residuals:

$$r_i(\theta_i) = \sqrt{\theta_i^2 - 10 \cos(2\pi\theta_i) + 10}$$

so that the optimization function can be written as:

$$f(\theta) = \sum_{i=1}^n r_i(\theta_i)^2$$

This residual formulation gives LMA an n -dimensional Jacobian with one row per parameter dimension, allowing it to compute gradient information and converge from a good starting point supplied by the WOA phase of the hybrid algorithm.

To evaluate the performance of the WOA-LMA hybrid algorithm, the standalone WOA, WOA-Lévy variant, LMA, and hybrid algorithms were applied to the Rastrigin function across 200 random initialization seeds. We recall that the diversity of the population at iteration t is

$$\text{Diversity}(t) = \frac{1}{D} \sum_{d=1}^D \frac{1}{N} \sum_{i=1}^N |\text{median}\{x_d(t)\} - x_{id}(t)|$$

The LMA phase of the hybrid algorithm was triggered once this quantity fell to 5% of its initial value, at which point the 10 best candidates identified by WOA were passed to LMA as starting points (the minimum distance filter d_{min} was set to 0.01). The diversity threshold was chosen to ensure that the population had sufficiently converged toward a promising basin before local

Algorithm	Mean Obj.	Median Obj.	Min Obj.	Max Obj.	Failed	$n < 0.1$	Mean Acc.	Median Acc.	Min Acc.	Max Acc.
LMA with random starts	1.14×10^1	1.09×10^1	1.99×10^0	2.69×10^1	0	0	3.27×10^0	3.30×10^0	1.41×10^0	5.17×10^0
WOA	0	0	0	0	0	200	5.43×10^{-9}	5.53×10^{-9}	1.99×10^{-9}	7.73×10^{-9}
WOA Lévy	3.82×10^0	2.99×10^0	1.86×10^{-5}	2.69×10^1	0	10	1.81×10^0	1.72×10^0	3.06×10^{-4}	5.17×10^0
WOA-LMA Hybrid	9.67×10^{-9}	6.17×10^{-10}	1.14×10^{-13}	1.26×10^{-7}	0	200	4.41×10^{-6}	1.76×10^{-6}	2.45×10^{-8}	2.52×10^{-5}

Table 4.2: Summary of algorithm performance on the Rastrigin problem across 200 seeds. Obj. refers to the objective value; Acc. refers to the Euclidean distance from the true solution (parameter accuracy). $n < 0.1$ is the number of seeds achieving an objective value below 0.1.

refinement was applied. Table 4.2 and Figure 4.5 show the accuracy of the parameter estimates achieved by each algorithm across all 200 seeds.

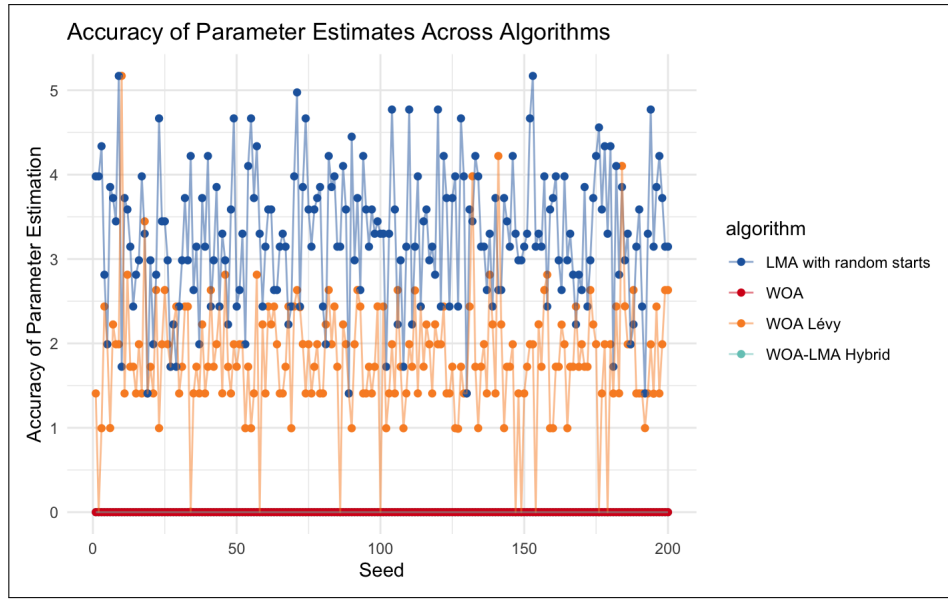


Figure 4.5: Accuracy of parameter estimates over 200 seeds

Recall the true parameter value in each dimension is zero. The accuracy was calculated by finding the Euclidean distance between each vector of parameter estimates and the true vector of all zeros. The results indicate that the hybrid algorithm achieves an accuracy comparable to standalone WOA, with a mean Euclidean distance 4.410^{-6} vs. 5.410^{-9} for WOA, confirming that the WOA phase in the hybrid algorithm reliably identifies the global basin on the Rastrigin function before the diversity threshold is met, allowing LMA to converge to the same minimum. Both algorithms appear to achieve perfect accuracy on the plot, however their differences can be seen in

Figure 4.6. In contrast, LMA with random starts and WOA Lévy struggle considerably on this problem, with mean Euclidean distances of 3.27 and 1.81 respectively, and neither reliably finds the global minimum; WOA Lévy finds a solution with objective value below 0.1 in only 10 of 200 seeds, while LMA with random starts finds none. The accuracy of the hybrid algorithm is still marginally worse than the WOA which is shown in Figure 4.6.

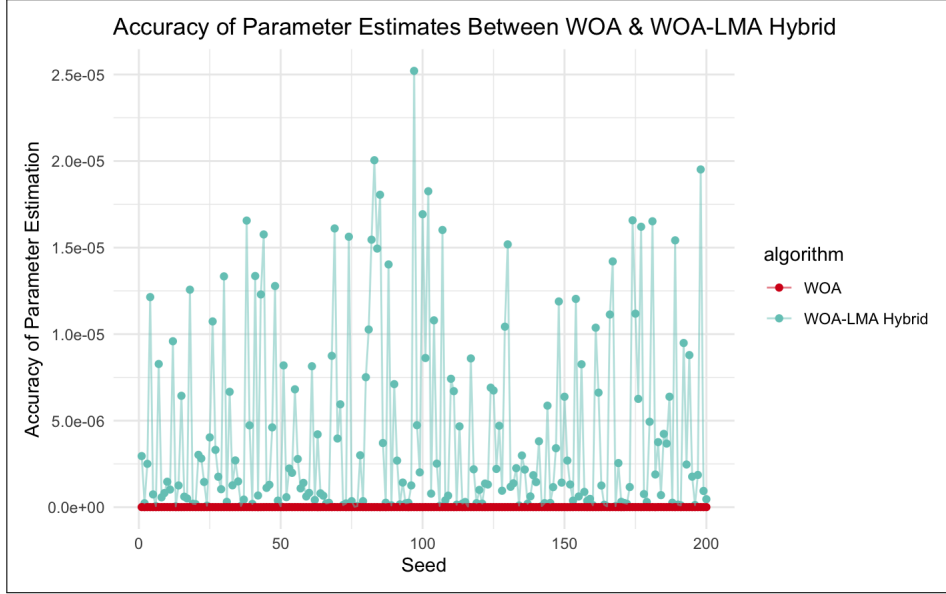


Figure 4.6: Accuracy of parameter estimates for WOA vs. Hybrid algorithm

Both WOA and the WOA-LMA Hybrid achieve near-perfect parameter estimation on the Rastrigin problem. WOA attains a mean Euclidean distance of 5.410^{-9} from the true solution across 200 seeds, and the Hybrid attains a mean of 4.410^{-6} , with a worst-case distance of 2.510^{-5} . Depending on the precision required in a true research problem, the small difference in accuracy between the algorithms is potentially negligible. The marginally larger errors in the hybrid algorithm are possibly attributable to numerical tolerance rather than genuine inaccuracy. LMA does not actually take derivatives, see Section 2.2.1, but rather uses numerical approximation to calculate the Jacobian, which terminates once the steps size falls below the threshold $\text{ptol} = \text{ftol} = 1\text{e-}8$ which are hyperparameters to the WOA-LMA Hybrid algorithm. The minute compromise in accuracy is benefited by a significant decrease in computation time.

Algorithm	Time (sec)
WOA Lévy	1.249
LMA with random starts	0.010
WOA-LMA Hybrid	0.160
WOA	0.608

Table 4.3: Average computation time per seed across 200 seeds.

With respect to computational efficiency, the total time across all 200 seeds was 121 seconds (0.608 sec/seed) for WOA and 32 seconds (0.160 sec/seed) for the WOA-LMA Hybrid algorithm. The hybrid was therefore approximately 73.6% faster in total, taking approximately 26.4% of the WOA computation time. This improvement is explained by the rapid convergence of LMA from the seeds supplied by WOA, avoiding the additional WOA iterations that would otherwise be required to achieve equivalent fitness values. The combination of a population-based global search phase and a gradient-informed local refinement phase thus retains the accuracy of WOA while reducing unnecessary computation in the later stages of optimization.

Chapter 5

Conclusion and Future Results

The promising performance of the hybrid algorithm on the Rastrigin benchmark suggests that combining the global exploration of a stochastic algorithm with the local refinement of a gradient based method can lower computation time. In particular, the observed reduction in computation time from the Whale Optimization Algorithm (WOA) to the hybrid approach indicates that incorporating a local method such as the Levenberg–Marquardt algorithm can accelerate convergence once the search is near a high-quality region of the parameter space. Given that the Rastrigin function is highly multimodal and widely used to evaluate global optimization methods, these results provide encouraging evidence that the hybrid strategy could perform well on similar optimization problems. However, performance on this single benchmark is not sufficient to establish any general conclusions. Future work should therefore evaluate the hybrid algorithm across a broader set of standard test functions, with varying degrees of modality, and dimensionality. One important note is that some other benchmark functions cannot be rewritten as the sum of squared residuals, and thus will not be meaningful test for the hybrid algorithm. This is also an important caveat in understanding which problems are ultimately a good match for this algorithm. Further experiments would help determine whether the observed improvements generalize beyond the specific structure of the Rastrigin landscape. Additionally, applying the method to real-world optimization problems—particularly those involving nonlinear least squares or similarly structured objective functions—would provide a more meaningful assessment of the hybrid algorithms practical utility. It is plausible that there exist application domains in which the hybrid approach outperforms standalone metaheuristic methods like WOA, especially in set-

tings where rapid local convergence could greatly decrease computation time after an initial global search phase. Identifying and validating such problems remains an important direction for future research.

Acknowledgments

Thank you first and foremost to Professor Jo Hardin for all of her help and support with this thesis, and for advising me through my adventures in math. Thank you to all of my professors and peers in the Pomona College math department for creating such a wonderful community that I get to be a part of, and thank you to my friends and family for all of their love.

Technology Statement

Generative AI tools were used to help generate code for the simulations in this thesis and to help with the formatting of this document. All simulations were done in R.

Bibliography

- Karrar Hameed Abdulkareem, Mazin Abed Mohammed, Zaid Abdi Alkareem Alyasseri, Dawood Zahi Khutar, and Osama Ahmad Alomari. Woa-covid-19: Whale optimization algorithm for selection of multi-examination features based on covid-19 infections. *Mesopotamian Journal of Computer Science*, 2025:172–185, 2025. doi: 10.58496/MJCSC/2025/010. Figure 1: “Approach of Humpback whales engage in bubble-net hunting” (ResearchGate).
- Josephine Adams, Johnson Hoang, Emily Petroni, Ethan Ashby, Johanna Hardin, and Daniel M. Stoebel. The timing of transcription of *rpoS*-dependent genes varies across multiple stresses in *Escherichia coli* k-12. *mSystems*, 8(5):e00663–23, 2023. doi: 10.1128/msystems.00663-23.
- M. Umut Caglar and Claus O. Wilke. *sicegar: Analysis of Single-Cell Viral Growth Curves*, 2021. URL <https://CRAN.R-project.org/package=sicegar>. R package version 1.2.1.
- M. Umut Caglar, Ashley I. Teufel, and Claus O. Wilke. Sicegar: R package for sigmoidal and double-sigmoidal curve fitting. *PeerJ*, 6:e4251, 2018. doi: 10.7717/peerj.4251.
- Augustin-Louis Cauchy. Méthode générale pour la résolution des systèmes d’équations simultanées. *Comptes Rendus de l’Académie des Sciences de Paris*, 25:536–538, 1847.
- Shikun Chen, Zebin Huang, and Wenlong Zheng. Optimization of non-smooth functions via differentiable surrogates. *PLOS ONE*, 20(3): e0321862, 2025. doi: 10.1371/journal.pone.0321862.
- Diegotorquemada. Rastrigin function. <https://commons.wikimedia.org/>

- `wiki/File:Rastrigin_function.png`, 2010. Wikimedia Commons, public domain, accessed April 14, 2026.
- Hangqi Ding, Zhiyong Wu, and Luchen Zhao. Whale optimization algorithm based on nonlinear convergence factor and chaotic inertial weight. *Concurrency and Computation: Practice and Experience*, 33(20):e5949, 2020. doi: 10.1002/cpe.5949. First published: August 10, 2020.
- Marco Dorigo, Vittorio Maniezzo, and Alberto Coloni. Distributed optimization by ant colonies. In *Proceedings of the First European Conference on Artificial Life*, pages 134–142. Elsevier, 1991. URL <https://staff.washington.edu/paymana/swarm/dorigo99-cec.pdf>.
- Timur V. Elzhov, Katharine M. Mullen, Andrej-Nikolai Spiess, and Ben Bolker. *minpack.lm: R Interface to the Levenberg-Marquardt Nonlinear Least-Squares Algorithm Found in MINPACK*, 2023. URL <https://CRAN.R-project.org/package=minpack.lm>. R package version 1.2-4.
- Henri P. Gavin. The levenberg–marquardt algorithm for nonlinear least squares curve-fitting problems. Technical manuscript, Department of Civil and Environmental Engineering, Duke University, 2024. URL <https://people.duke.edu/~hpgavin/m-files/lm.pdf>. Available at <https://people.duke.edu/~hpgavin/m-files/lm.pdf>.
- James Kennedy and Russell C. Eberhart. Particle swarm optimization. In *Proceedings of the IEEE International Conference on Neural Networks*, volume IV, pages 1942–1948. IEEE, 1995. doi: 10.1109/ICNN.1995.488968. URL <https://ieeexplore.ieee.org/document/488968>.
- Kenneth Levenberg. A method for the solution of certain problems in least squares. *Quarterly of Applied Mathematics*, 2:164–168, 1944.
- Paul Lévy. *Calcul des probabilités*. Gauthier-Villars, Paris, 1925.
- Ying Ling, Yongquan Zhou, and Qifang Luo. Lévy flight trajectory-based whale optimization algorithm for global optimization. *IEEE Access*, 5: 6168–6186, 2017. ISSN 2169-3536. doi: 10.1109/ACCESS.2017.2695498.
- Janet Mann, Richard C. Connor, Peter L. Tyack, and Hal Whitehead, editors. *Cetacean Societies: Field Studies of Dolphins and Whales*. University of Chicago Press, Chicago, IL, USA, 2000. ISBN 0226503410, 9780226503417.

- Rosario Nunzio Mantegna. Fast, accurate algorithm for numerical simulation of lévy stable stochastic processes. *Physical Review E*, 49(5):4677–4683, 1994. doi: 10.1103/PhysRevE.49.4677.
- Donald W. Marquardt. An algorithm for least-squares estimation of nonlinear parameters. *SIAM Journal on Applied Mathematics*, 11(2):431–441, 1963.
- Taseer Mehboob. Understanding gradient descent: The optimization algorithm behind machine learning, June 2024. Medium blog post, accessed 2025-11-14.
- Seyedali Mirjalili and Andrew Lewis. The whale optimization algorithm. *Advances in Engineering Software*, 95:51–67, 2016. ISSN 0965-9978. doi: 10.1016/j.advengsoft.2016.01.008.
- M.H. Nadimi-Shahraki, H. Zamani, Z. Asghari Varzaneh, and S. Mirjalili. A systematic review of the whale optimization algorithm: Theoretical foundation, improvements, and hybridizations. *Archives of Computational Methods in Engineering*, pages 1–47, 2023. doi: 10.1007/s11831-023-09928-7. Epub ahead of print, May 27, 2023.
- John P. Nolan. Chapter 1: Stable distributions. <https://edspace.american.edu/jpnolan/wp-content/uploads/sites/1720/2020/09/Chap1.pdf>, 2020.
- R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2025. URL <https://www.R-project.org/>.
- L. A. Rastrigin. *Systems of Extremal Control*. Mir Publishers, Moscow, 1974.
- Lala Septem Riza, Iip, Eddy Prasetyo Nugroho, Muhammad Bima Adi Prabowo, Enjun Junaeti, and Ade Gafar Abdullah. *metaheuristicOpt: Metaheuristic Optimization Algorithms*, 2024. URL <https://cran.r-project.org/package=metaheuristicOpt>. R package version 2.0.0.
- Mark K Transtrum, Benjamin B Machta, and James P Sethna. Geometry of nonlinear least squares with applications to sloppy models and optimization. *Physical Review E*, 83(3):036701, 2011.
- Vladimir M. Zolotarev. *One-Dimensional Stable Distributions*. American Mathematical Society, Providence, RI, 1986.